



# 2026 Coding Standards Uni Textbook Marketplace

For **Demo 2**

Presented by:



# Coding Standards

**Project:** Uni Textbook Marketplace  
**Team:** NexusDev  
**Client:** Agile Bridge  
**University:** University of Pretoria - COS 301 Software Engineering  
2026  
**Document version:** 1.0 - Demo 2  
**Last updated:** 30 July 2026

---

## The Team

| Team Member        | Student number |
|--------------------|----------------|
| 1. Tiego Mokwena*  | 22496336       |
| 2. Gift Mohuba     | 23545527       |
| 3. Neo Bosoga      | 23591732       |
| 4. Josh Kretschmer | 22883925       |

---

# Contents

---

|          |                                            |           |
|----------|--------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                        | <b>3</b>  |
| <b>2</b> | <b>Repository File Structure</b>           | <b>4</b>  |
| 2.1      | Module Structure (Backend) . . . . .       | 5         |
| 2.2      | Route Structure (Frontend) . . . . .       | 5         |
| <b>3</b> | <b>Naming Conventions</b>                  | <b>6</b>  |
| <b>4</b> | <b>TypeScript and Formatting Standards</b> | <b>7</b>  |
| 4.1      | Compiler Configuration . . . . .           | 7         |
| 4.2      | Linting . . . . .                          | 7         |
| 4.3      | Static Analysis . . . . .                  | 7         |
| <b>5</b> | <b>Backend Standards (NestJS)</b>          | <b>8</b>  |
| 5.1      | Layering . . . . .                         | 8         |
| 5.2      | Guards and Authorization . . . . .         | 8         |
| 5.3      | Data Transfer Objects . . . . .            | 8         |
| 5.4      | Database Migrations . . . . .              | 8         |
| <b>6</b> | <b>Frontend Standards (Next.js)</b>        | <b>10</b> |
| 6.1      | Styling . . . . .                          | 10        |
| 6.2      | Authentication State . . . . .             | 10        |
| 6.3      | API Calls . . . . .                        | 10        |
| <b>7</b> | <b>Git and Version Control</b>             | <b>11</b> |
| 7.1      | Branching Model . . . . .                  | 11        |
| 7.2      | Branch Protection . . . . .                | 11        |
| 7.3      | Commit Messages . . . . .                  | 11        |
| <b>8</b> | <b>Testing Standards</b>                   | <b>12</b> |
| <b>9</b> | <b>Environment Variables and Secrets</b>   | <b>13</b> |

## 1. Introduction

---

This document lays out the coding conventions we use across the Uni Textbook Marketplace project. The main point is just to keep things consistent, since four people are writing code and it's easy for that to turn into a mess if everyone does their own thing. It should also help whoever's marking this actually follow the repo without us having to explain everything in person.

Most of what's below isn't just theory, a lot of it is actually enforced by ESLint, SonarCloud, and our CI pipeline (more detail on that in the SAS document), so if you break one of the bigger rules the pull request literally won't merge.

## 2. Repository File Structure

We're using a monorepo, so it's one Git repo with the backend and frontend sitting in their own top-level folders, plus the CI/CD config.

```
Uni-Textbook-Marketplace/
.github/
  workflows/
    ci.yml          # lint, test, coverage, on every push and PR
    deploy.yml      # build and deploy, branch-conditional
backend/
  src/
    auth/           # AuthController, AuthService, guards, DTOs
    listings/       # Listing CRUD, search, edit/withdraw
    admin/          # Admin dashboard, approve/reject listings
    modules/        # Module search-as-you-type
    books/          # Book/catalogue metadata
    messaging/      # Buyer-seller messaging (Firestore-backed)
    notifications/  # Cross-cutting notification/alert system
    saved_search/   # Saved search feature
    wishlist/       # Wishlist feature
    azure/          # Blob upload controller
    email/          # Email sending (OTP, alerts)
    firebase/       # Firebase Admin SDK init (messaging)
    shared/         # Shared providers/utilities across modules
    database/
      entities/     # TypeORM entity classes
      migrations/   # TypeORM migration files, flat, no nesting
      seeds/        # Idempotent seed scripts
    data-source.ts  # TypeORM CLI data source
    main.ts         # Application bootstrap, CORS config
    app.module.ts   # Root module, TypeOrmModule config
  test/            # e2e tests (Supertest)
Dockerfile
package.json
frontend/
  src/
    app/            # Next.js App Router, one folder per route
    components/     # Reusable UI components
    context/        # React context providers (e.g. AuthContext)
    hooks/          # Custom React hooks
    lib/            # API client, utility functions
```

```
providers/           # Theme and other app-wide providers
types/              # Shared TypeScript types
utils/              # Frontend utility functions
Dockerfile
package.json
database/
  schema/           # Reference DB schema
docs/               # SRS, SAS, Coding Standards, Testing Policy,
  wireframes
messaging/          # Firebase Firestore messaging microservice
docker-compose.yml  # Local dev environment
.env                # Local environment variables, not committed
```

## 2.1 Module Structure (Backend)

Each backend feature is its own self-contained NestJS module. Modules aren't allowed to import another module's service directly, if two modules need to share something, it goes through a shared provider instead of a random cross-import. A typical module looks like this:

```
listings/
  dto/
    create-listing.dto.ts
    update-listing.dto.ts
  listings.controller.ts
  listings.service.ts
  listings.module.ts
```

## 2.2 Route Structure (Frontend)

We're using the Next.js App Router, so every route gets its own folder under `src/app`, named after the URL it serves (e.g. `src/app/saved-searches/page.tsx` is `/saved-searches`). Anything only used on that one page lives next to it, and anything reused across pages goes into `src/components`.

### 3. Naming Conventions

| What                                  | Convention                       | Example                                                                                                         |
|---------------------------------------|----------------------------------|-----------------------------------------------------------------------------------------------------------------|
| TypeScript classes, interfaces, enums | PascalCase                       | <code>ListingService,</code><br><code>CreateListingDto</code>                                                   |
| Variables, functions, methods         | camelCase                        | <code>getPendingListings()</code>                                                                               |
| NestJS file names                     | kebab-case, dot-suffixed by role | <code>listings.controller.ts,</code><br><code>listings.service.ts,</code><br><code>create-listing.dto.ts</code> |
| React component files                 | PascalCase                       | <code>NavBar.tsx,</code><br><code>AuthContext.tsx</code>                                                        |
| Database tables and columns           | snake_case                       | <code>listings,</code> <code>module_id,</code><br><code>created_at</code>                                       |
| Environment variables                 | SCREAMING_SNAKE_CASE             | <code>DATABASE_URL,</code><br><code>JWT_ACCESS_SECRET</code>                                                    |
| Git branches                          | type/short-description           | <code>feature/saved-search-frontend,</code><br><code>fix/azure-fixes-and-faculty-loads</code>                   |

## 4. TypeScript and Formatting Standards

### 4.1 Compiler Configuration

Both the backend and frontend use fairly strict TypeScript settings. The main ones worth knowing about:

- `strictNullChecks` is on, so you have to actually handle `null/undefined` instead of just assuming a value will be there.
- `forceConsistentCasingInFileNames` is on. This mostly matters because Windows/-macOS don't care about file name casing but Linux does, so an import with the wrong case can pass locally and still break the build.
- Class properties that never get reassigned should be marked `readonly`, so if you accidentally try to reassign one, the compiler catches it instead of it becoming a runtime bug.

### 4.2 Linting

ESLint runs in `ci.yml` on every push and PR to `main` and `develop`, separately for backend and frontend. If lint fails, the `ci-success` check fails, and branch protection won't let the PR merge until it's fixed.

**Something we learned the hard way:** running `eslint -fix` across the whole repo mid-sprint also touches files you weren't even working on, including stuff a teammate had half-finished. Only stage and commit the files you actually meant to change, and `git checkout - <file>` the rest instead of committing a giant auto-fix.

### 4.3 Static Analysis

SonarCloud also runs on every push/PR to `main` and `develop` and checks for code smells, security hotspots, and coverage. One thing that came out of an early SonarCloud finding: we now pin GitHub Actions to full commit SHAs instead of version tags in every workflow file, since tags can technically be moved.

## 5. Backend Standards (NestJS)

---

### 5.1 Layering

Controllers only handle HTTP stuff, routing, guards, DTO validation. All the actual logic lives in services, and services never write raw SQL, everything goes through TypeORM repositories. This way you can read a controller top to bottom and know what an endpoint does without having to dig through query logic.

### 5.2 Guards and Authorization

Every protected route uses `@UseGuards(JwtAuthGuard, RolesGuard)`. We don't hand-roll auth checks inside controller methods, it always goes through guards so the same check behaves the same way everywhere.

### 5.3 Data Transfer Objects

Every request body gets its own DTO class. The DTO is the source of truth for what an endpoint accepts, and validation decorators live there, not scattered through the service.

### 5.4 Database Migrations

Migrations have honestly caused us more real bugs than anything else in this project, so these rules exist because of specific things that went wrong, not just because they sounded like good practice:

- **Migration files live directly in `backend/src/database/migrations/`, never in a subfolder.** We once had a nested `migrations/src/migrations/` folder and it silently broke the compile into `dist/`, because it messed with TypeScript's implicit `rootDir` guess.
- **The migration class name has to end with the exact same timestamp as the filename.** So for `1784402940000-AddSavedSearches.ts`, the class needs to be `AddSavedSearches1784402940000`. TypeORM only checks this at runtime, not when it compiles, so it looks fine until the migration actually tries to run.
- **Don't add a `name = '...'` property to a migration class if you don't need to.** TypeORM checks that property first if it exists, and it'll silently override the correct class name even after you've already fixed the class name. This is how the same migration failed twice in a row for us, once for the class name and then again for basically the same reason in disguise.

- **Double check every column name in a migration's raw SQL against the actual entity, don't just assume.** We once wrote `module` in an index when the real foreign key column (auto-generated by TypeORM from the `@ManyToOne` relation) was actually `module_id`.
- **Run `npm run build` and check the migration actually shows up in `dist/database/migrations` before assuming a fix worked.** A fix that only exists in `src/` does nothing in a deployed container, since that runs the compiled `dist/` output.
- Seed scripts (`university.seed.ts`, `faculty.seed.ts`) check if a row already exists before inserting, so you can re-run them safely against a DB that's already partly seeded without ending up with duplicates.

## 6. Frontend Standards (Next.js)

---

### 6.1 Styling

We use Tailwind everywhere, utility classes right in the JSX instead of separate CSS files. This keeps a component's styling visible next to its markup, and using the token-based colours keeps everything on-brand with the Agile Bridge style guide instead of random one-off colours creeping in.

### 6.2 Authentication State

Auth state lives in one `AuthContext`, mounted once in the root layout. Which routes need login is just an explicit list (public vs. protected) checked against the current path, not a bunch of scattered per-page checks. If you add a new public page (like a help page), you have to add it to that list or it'll redirect people to login who shouldn't need to. This actually happened to us during Demo 2 prep, the Help and Brand Style Guide links in the footer were wrongly requiring login.

### 6.3 API Calls

All backend calls go through one shared client in `src/lib` instead of components calling `fetch` directly. That way the base URL, credentials, and error handling only need to be right in one place.

## 7. Git and Version Control

### 7.1 Branching Model

- `main` is always what's actually in production.
- `develop` is always what's on staging.
- All work happens on a feature branch off `develop`, named `type/short-description` (e.g. `feature/saved-search-frontend`, `fix/azure-fixes-and-faculty-loads`).
- Feature branches merge into `develop` through a PR. `develop` merges into `main` the same way, once we're sure it's working on staging.

### 7.2 Branch Protection

Both `main` and `develop` have branch protection on:

- No direct pushes, everything has to come through a PR from a feature branch.
- At least one other team member has to review and approve it.
- The `ci-success` check (backend + frontend lint/test jobs plus SonarCloud) has to pass, or the merge is blocked.

### 7.3 Commit Messages

We try to follow Conventional Commits, so `type: short description`:

| Type                  | Used for                                               |
|-----------------------|--------------------------------------------------------|
| <code>feat</code>     | A new feature                                          |
| <code>fix</code>      | A bug fix                                              |
| <code>ci</code>       | Changes to GitHub Actions workflows or pipeline config |
| <code>docs</code>     | Documentation-only changes                             |
| <code>refactor</code> | Restructuring code without changing behaviour          |

**Something we agreed on:** commit a working fix straight away instead of leaving it staged. We lost an uncommitted fix once to an accidental `git checkout` and had to redo the whole thing.

## 8. Testing Standards

---

Backend tests run with Jest against a real, temporary Postgres container that spins up inside CI, not a mocked DB connection, so a passing test suite actually means something rather than just testing logic in isolation. Frontend tests are Jest unit tests for now, we're planning to add Cypress end-to-end tests but haven't gotten there yet. Coverage for both sides is tracked through Codecov, with separate flags for backend and frontend.

## 9. Environment Variables and Secrets

---

- No credentials, connection strings, or API keys ever get committed, in any file, at any point.
- Locally, variables go in a `.env` file at the repo root, which `docker-compose.yml` pulls in through `$VAR_NAME`. It's in `.gitignore` and never committed.
- In deployed environments, variables are stored as Azure Container Apps secrets (`secretref:`) and injected at the container level, or as GitHub Actions secrets for anything CI/CD itself needs (like `AZURE_CREDENTIALS`).
- A JSON-formatted secret (like the Firebase service account key) is stored as one env variable holding the whole JSON string, and parsed at runtime with `JSON.parse()`, instead of being a file, since an actual credential file should never be sitting in the repo.

# Uni Textbook Marketplace

In collaboration **with:**



2026 NexusDev - University of Pretoria - COS 301 Capstone Project  
This document is version-controlled in the GitHub repository under `/docs/Demo`  
`2/Coding Standards.pdf`